⚙️ INSTALLATION GUIDE — UNIVERSAL SPEC ARCHITECT MCP SERVER

Installing the MCP Server

A step-by-step guide to download, install, and verify the Universal Spec Architect MCP Server on Windows, macOS, and Linux — ready to connect to any AI coding assistant.

🖥️ Windows · macOS · Linux

⏱️ Setup Time: ~15 minutes

🐍 Requires: Python 3.9+

1 Prerequisites → 2 Download → 3 Install Python Deps → 4 Configure Assistant → 5 Verify Connection → 6 Run First Spec

WHAT YOU WILL INSTALL

The Universal Spec Architect MCP Server is a lightweight Python process that runs locally on your machine. Your AI coding assistant connects to it over stdio and gains access to 6 spec-driven tools. Once installed, it serves all your projects — you never install it twice.

🤖 **Coding Assistant** — Bob / Cursor / Cline

↔

⚙️ **MCP Server** — `universal_spec_mcp`

→

📄 **Spec Files** — `.specs/{feature}/`

1 Check Prerequisites

Verify your system has everything needed before downloading

📋 Required Software

SOFTWARE	MINIMUM VERSION	REQUIRED?	CHECK COMMAND
Python	3.9 or higher	Required	`python --version`
pip	21.0 or higher	Required	`pip --version`
Git	2.x	Optional	`git --version`

SOFTWARE	MINIMUM VERSION	REQUIRED?	CHECK COMMAND
AI Coding Assistant	Any MCP-compatible version	Required	IBM Bob / Cursor / Cline / Windsurf

⚠️ **Windows users:** If `python --version` returns an error, try `python3 --version`. If neither works, install Python from **python.org** and tick **"Add Python to PATH"** during installation.

🐍 Install Python (if not already installed)

🪟 Windows

PowerShell (Run as Administrator)

```
# Option 1 — Download installer from python.org (recommended)
# Visit https://python.org/downloads and run the installer
# IMPORTANT: tick "Add Python to PATH" before clicking Install

# Option 2 — Install via winget
PS> winget install Python.Python.3.11

# Verify
PS> python --version
Python 3.11.x
```

🍎 macOS

Terminal

```
# Install via Homebrew (recommended)
$ brew install python@3.11

# Or download from python.org/downloads/macos

# Verify
$ python3 --version
Python 3.11.x
```

🐧 Linux

Terminal (Ubuntu / Debian)

```
$ sudo apt update && sudo apt install python3.11 python3-pip -y

# Verify
$ python3 --version
Python 3.11.x
```

2 Download the MCP Server

Get the server files onto your machine in a permanent shared location

📦 Choose a Permanent Location

The MCP server is a **shared tool** — like a compiler. It lives outside your projects and serves all of them. Choose a permanent folder that will not be deleted.

📁 Windows: **C:\Tools\universal-spec-mcp**

📁 macOS / Linux: **~/tools/universal-spec-mcp/**

⚠️ **Do not** put the MCP server inside your project folder. It is shared across all your projects.

⬇️ Option A — Clone from GitHub (Recommended)

🪟 Windows

PowerShell

```
PS> mkdir C:\Tools
PS> cd C:\Tools
PS> git clone https://github.com/ahmedelpannann/universal-spec-mcp.git
Cloning into 'universal-spec-mcp'...
done.
```

🍎 macOS 🐧 Linux

Terminal

```
$ mkdir -p ~/tools && cd ~/tools
$ git clone https://github.com/ahmedelpannann/universal-spec-mcp.git
Cloning into 'universal-spec-mcp'...
done.
```

📁 Option B — Download ZIP (No Git Required)

Go to `github.com/ahmedelpannann/universal-spec-mcp`, click the green **Code** button, then click **Download ZIP**. Extract the zip to your chosen location.

🪟 Windows

```
PowerShell — Extract ZIP

PS> Expand-Archive -Path "$env:USERPROFILE\Downloads\universal-spec-mcp.zip" -DestinationPath "C:\Tools\"
# Or right-click the zip → Extract All → choose C:\Tools\
```

🍎 macOS 🐧 Linux

```
Terminal

$ mkdir -p ~/tools && cd ~/tools
$ unzip ~/Downloads/universal-spec-mcp.zip
```

3 Install Python Dependencies

Install the two required packages: FastMCP and Pydantic

⚙️ Install with pip

🪟 Windows

```
PowerShell

PS> cd C:\Tools\universal-spec-mcp
PS> pip install fastmcp pydantic
Successfully installed fastmcp pydantic
```

🍎 macOS 🐧 Linux

```
Terminal

$ cd ~/tools/universal-spec-mcp
$ pip3 install fastmcp pydantic
Successfully installed fastmcp pydantic
```

> 💡 If you use a virtual environment, activate it first before running pip. If you get a permissions error on Linux/macOS, add `--user` to the pip command.

✅ Verify the Server Starts Correctly

🪟 Windows

PowerShell — Quick smoke test

```
PS> python C:\Tools\universal-spec-mcp\src\universal_spec_mcp\server.py --help
Universal Spec Architect MCP Server
6 tools registered: initialize_spec, write_requirements, ...
```

🍎 macOS 🐧 Linux

Terminal

```
$ python3 ~/tools/universal-spec-mcp/src/universal_spec_mcp/server.py --help
Universal Spec Architect MCP Server
6 tools registered: initialize_spec, write_requirements, ...
```

> ✅ If you see the server output without errors, the installation is complete. You are ready to connect it to your coding assistant.

4 Connect to Your Coding Assistant

Register the MCP server with your preferred AI coding assistant

🤖 IBM Bob

Copy the `.bob/` folder from the extracted zip into your project root. Then open `.bob/mcp.json` and update the path.

.bob/mcp.json

```
{
  "mcpServers": {
    "universal-spec-architect": {
      "command": "python",
      "args": [
        "C:/Tools/universal-spec-mcp/src/universal_spec_mcp/server.py"
      ],
      "env": {
        "PYTHONPATH": "C:/Tools/universal-spec-mcp/src"
      }
    }
  }
}
```

💡 Replace `C:/Tools/universal-spec-mcp` with your actual path. On macOS/Linux use `/home/yourname/tools/universal-spec-mcp`. Use forward slashes even on Windows.

🖱️ Cursor IDE

Create a file at `.cursor/mcp.json` in your project root with the following content:

.cursor/mcp.json

```
{
  "mcpServers": {
    "universal-spec-architect": {
      "command": "python",
      "args": [
        "C:/Tools/universal-spec-mcp/src/universal_spec_mcp/server.py"
      ],
      "env": {
        "PYTHONPATH": "C:/Tools/universal-spec-mcp/src"
      }
    }
  }
}
```

💜 After saving, open Cursor Settings → MCP → you should see **universal-spec-architect** listed as Connected.

💙 VS Code (GitHub Copilot Agent Mode)

Open the global MCP config file at `%APPDATA%\Code\User\mcp.json` (Windows) or `~/.config/Code/User/mcp.json` (macOS/Linux) and add the server entry:

mcp.json — VS Code Global Config

```
{
  "servers": {
    "universal-spec-architect": {
      "type": "stdio",
      "command": "python",
      "args": [
        "C:/Tools/universal-spec-mcp/src/universal_spec_mcp/server.py"
      ],
      "env": {
        "PYTHONPATH": "C:/Tools/universal-spec-mcp/src"
      }
    }
  }
}
```

🔌 Cline (VS Code Extension)

In VS Code, open the Cline panel → click the **MCP Servers** icon → click **Edit MCP Settings** and paste the following:

Cline MCP Settings

```
{
  "universal-spec-architect": {
    "command": "python",
    "args": [
      "C:/Tools/universal-spec-mcp/src/universal_spec_mcp/server.py"
    ],
    "env": {
      "PYTHONPATH": "C:/Tools/universal-spec-mcp/src"
    },
    "disabled": false,
    "autoApprove": []
  }
}
```

Windsurf IDE

Open `~/.codeium/windsurf/mcp_config.json` and add the server entry under `mcpServers`:

~/.codeium/windsurf/mcp_config.json

```
{
  "mcpServers": {
    "universal-spec-architect": {
      "command": "python3",
      "args": [
        "/home/yourname/tools/universal-spec-mcp/src/universal_spec_mcp/server.py"
      ],
      "env": {
        "PYTHONPATH": "/home/yourname/tools/universal-spec-mcp/src"
      }
    }
  }
}
```

5 Verify the Connection

Confirm all 6 tools are visible and connected in your assistant

🔍 Check the MCP Tools Panel

After saving your config file, restart your coding assistant. Then open the MCP tools panel and confirm all 6 tools show as **Connected**:

📋 initialize_spec ✅ Connected	📝 write_requirements ✅ Connected	🏗️ write_design ✅ Connected
✅ write_tasks ✅ Connected	🔄 update_task_status ✅ Connected	🪝 run_hook ✅ Connected

⚠️ **If tools show as Disconnected:** Double-check the file path in your config — it must point exactly to `server.py`. On Windows, use forward slashes. On macOS/Linux, use the full absolute path starting with `/`.

🛠️ Common Connection Issues

- ❌ **Error: python not found** — On macOS/Linux, change `"command": "python"` to `"command": "python3"` in your config file.
- ❌ **Error: No module named fastmcp** — Run `pip install fastmcp pydantic` again. If using multiple Python versions, make sure you installed into the same Python that your config points to.
- ❌ **Error: File not found** — The path in your config is wrong. Open a terminal, navigate to the server folder, and run `pwd` (macOS/Linux) or `cd` (Windows) to get the exact path.
- ❌ **Tools show but don't respond** — Restart your coding assistant completely after saving the config file. Some assistants require a full restart to pick up new MCP servers.

6 Run Your First Spec

Test the full workflow with a single message to your assistant

💬 Send This Message to Your Assistant

In your assistant's chat panel (in Agent mode), type:

```
Chat Message — Agent Mode

I want to build a user login feature. Use the spec-driven workflow.
```

Your assistant will automatically call all 4 tools in sequence:

`initialize_spec`	Creates `.specs/user-login/` directory in your project
`write_requirements`	Generates `requirements.md` with EARS-validated requirements
`write_design`	Generates `design.md` with architecture and data models
`write_tasks`	Generates `tasks.md` with ordered, dependency-linked tasks

> ✅ After the 4 tool calls complete, open your project in the file explorer. You will see a new `.specs/user-login/` folder with three files. **No code has been written yet** — that is the point.

🔁 What Happens Next

Review the three generated spec files. Once you are satisfied, tell your assistant to begin implementation:

```
Chat Message

Implement TASK-001 from the user-login spec.
```

The assistant will fire the `run_hook("pre_task")` to verify spec files exist, write the code, then fire `run_hook("post_task")` and call `update_task_status` to mark the task as complete.

📚 The 6 MCP Tools — Quick Reference

`initialize_spec`	Creates `.specs/{feature}/` directory and sets the workflow variant (requirements-first or design-first)
`write_requirements`	Drafts `requirements.md` — rejects any requirement not in EARS notation (`WHEN... THE SYSTEM SHALL...`)
`write_design`	Drafts `design.md` with mandatory sections: Architecture Overview, Sequence Diagrams, Data Models, Error Handling
`write_tasks`	Drafts `tasks.md` with discrete, ordered tasks that include explicit dependency links
`update_task_status`	Updates task status in real time: `todo` → `in_progress` → `completed`
`run_hook`	Fires automation hooks: `pre_task` (verify specs), `post_task` (run tests), `post_save` (lint)

📂 Files Created Per Feature

Project structure after running the workflow

```
my-project/
├── .bob/                          ← Bob config (steering + rules)
│   ├── mcp.json
│   ├── modes/spec-architect.json
│   ├── rules/spec-workflow.md
│   └── steering/
│       ├── product.md             ← fill in once
│       ├── tech.md                ← fill in once
│       └── structure.md           ← fill in once
├── .specs/                        ← generated by MCP server
│   └── user-login/
│       ├── requirements.md        ← EARS-validated
│       ├── design.md              ← architecture + models
│       └── tasks.md               ← tracked task list
└── src/                           ← your code goes here
```

🔗 Useful Links

📦 **GitHub Repository:** `github.com/ahmedelpannann/universal-spec-mcp`

📄 **IBM Bob From-Scratch Guide:** `docs/bob_from_scratch_guide.pdf`

📄 **Cline From-Scratch Guide:** `docs/new_project_from_scratch_guide.pdf`

📄 **End-to-End Use Case Walkthrough:** `docs/end_to_end_usecase_walkthrough.md`

🐍 **Python Download:** `python.org/downloads`

🔌 **Cline Extension:** Search `Cline` in VS Code Extensions marketplace

🎉 Installation Complete

The MCP server is installed and connected. Every feature you build now starts with a validated spec before a single line of code is written.

6	5	3
MCP tools installed	Coding assistants supported	Platforms covered

0
Lines of code before the spec

View on GitHub